Herramientas de Ingeniería: Sistema distribuido de reducción adaptativa de ruido

CE 1110 Análisis de Señales Mixtas | Jorge Pablo Porras Alvarado, Marcelo Matey Rodríguez, Josué Adrián Pérez Pérez

a) Selección de técnicas, recursos, herramientas y métodos

El problema exige reducir ruido desconocido en una señal contaminada, por bloques y en un microcontrolador de cómputo y memoria limitados, con dos métodos distintos y una comparación objetiva entre ellos. Las herramientas se eligieron según esas variables (los números entre corchetes remiten a las referencias del artículo):

Herramienta o método	Uso y motivo de la selección
FFT radix-2 con ventana de Hann	Análisis espectral del Nodo 1 y base del método espectral. $N = 1024$ (potencia de 2, exigida por el enunciado y por `ulab`) da $\Delta f = f_s/N = 15.6$ Hz y bloques de 32 ms; la ventana reduce la fuga espectral [2], [3].
Interpolación parabólica del pico	Refinar la frecuencia de la interferencia por debajo de Δf sin aumentar N ni la latencia.
Notch IIR de 2.° orden, pasaaltos IIR y FIR pasabajos (Hamming, 63 coef.)	Ruta temporal (Nodo 2): el notch anula un tono con solo 5 coeficientes y es adaptativo; el FIR de fase lineal elimina el ruido fuera de la banda útil [1].
Ganancia de Wiener por bin, notch espectral y *overlap-add*	Ruta espectral (Nodo 3): se adapta al ruido de banda ancha o solapado estimando el ruido en cada bloque [4]–[6].
Tramas con sincronía, n.° de bloque y CRC-16 CCITT	Detectar errores y bloques perdidos en la comunicación.
Raspberry Pi Pico W, MicroPython 1.24 con `ulab`, micrófono MAX4466	Plataforma del enunciado. `ulab` ejecuta la FFT y `sosfilt` en C, mucho más rápido que un ciclo interpretado [7]–[9].
Python 3 (NumPy, SciPy, Matplotlib, sounddevice, pyserial)	Simulación, referencias de validación (`scipy.signal`), métricas, figuras, reproducción del audio de prueba y comunicación USB con la Pico.
Git y GitHub, VS Code, Thonny, LaTeX (IEEEtran)	Desarrollo, control de versiones y documentación.
Asistente de IA (Claude, de Anthropic)	Ver la declaración de uso al final.
Fuentes consultadas	Oppenheim y Schafer (filtros, estabilidad), Harris (ventanas), Allen (*overlap-add*), Boll y Scalart (sustracción espectral y Wiener), hojas de datos del RP2040 y del MAX4466 y el libro de `ulab`. No se copió código de fuentes externas.

b) Cómo se aplican en el problema

Modelos y ecuaciones. Resolución $\Delta f = f_s/N$. Notch con $\omega_0 = 2\pi f_0/f_s$, $\beta = \tan(\omega_0/2Q)$, $g = 1/(1+\beta)$ y $Q = 10$:

$$H(z) = g\,\frac{1 - 2\cos\omega_0\, z^{-1} + z^{-2}}{1 - 2g\cos\omega_0\, z^{-1} + (2g-1)\, z^{-2}}.$$

Frecuencia estimada $\hat{f} = (k+p)f_s/N$ con $p = \frac{1}{2}(\alpha-\gamma)/(\alpha-2\beta+\gamma)$ sobre $\ln|X|$ en el máximo y sus vecinos. Ganancia $G = \mathrm{snr}/(\mathrm{snr}+1)$. Reconstrucción exacta del *overlap-add* porque $\sqrt{w}\cdot\sqrt{w}$ es una Hann periódica, cuyas mitades suman 1 con 50 % de solapamiento. Métricas: MSE, SNR, ΔSNR, energía útil conservada y atenuación de la interferencia.

Scripts creados o modificados.

- `simulacion_sistema.py` (PC): los tres nodos como objetos con estado entre bloques, tramas reales de 1034 B con CRC, un canal que altera tramas y un "oráculo" que calcula las métricas. Es el prototipo del firmware.
- `banco_pruebas.py` (PC): genera P1–P5 con semilla fija, SNR de 5 dB y tres repeticiones; corre las tres rutas (notch, notch con FIR y espectral) y guarda métricas en CSV, figuras y audios.
- `main_pico.py` (Pico): muestreo con temporizador a 16 kHz, eliminación de DC, los tres nodos con `ulab`, tramas con CRC por tabla y comandos `PING`, `SET`, `GET` y `REC`, que permiten cambiar parámetros sin reprogramar. Mide la latencia de cada nodo y la memoria libre.
- `pc_cliente.py` (PC): reproduce la prueba por el parlante, envía el comando de grabación, recibe y verifica las tramas, remuestrea la referencia al reloj de la Pico, la alinea por correlación cruzada y calcula métricas en frecuencia y por banda.
- `bench_pico.py` y `bench_pico_2.py` (Pico): tiempo de FFT/IFFT según N y la frecuencia de CPU, con el temporizador del ADC activo y en el segundo núcleo, para localizar el cuello de botella.
- Reutilizados de la Tarea 1: `filtrado_espectral.py` (estimación de ruido y ganancia) y `comparacion.py` (métricas). `emisor_tonos.py` y `receptor_tonos.py` sirvieron para la primera prueba de tonos.

Procedimiento. (1) Prototipo del sistema en simulación. (2) Validación de cada nodo contra una referencia independiente: coeficientes del notch frente a `scipy.signal.iirnotch` (error 2.2×10^{-16}), filtrado por bloques frente a la señal completa (error 0), *overlap-add* con $G = 1$ (error 5.6×10^{-16}) y 5 % de tramas alteradas, todas detectadas

por el CRC. (3) Migración al firmware con el mismo formato de trama. (4) Cinco pruebas con tres repeticiones en hardware y comparación del orden de los métodos con la simulación.

c) Adaptación de las técnicas al problema

- **Frecuencia de muestreo real.** El temporizador da 16 130–16 220 Hz, no 16 000. Los nodos trabajan en frecuencia normalizada (el Nodo 1 estima y el Nodo 2 diseña con la misma f_s), la Pico mide f_s en cada corrida y la PC remuestrea la referencia a ese reloj antes de comparar.
- **Resolución insuficiente.** Con $\Delta f = 15.6$ Hz el error de estimación llega a 7.8 Hz y el notch atenuaría solo unos 21 dB. La interpolación parabólica bajó el error medio de 7.5 a 0.05 Hz sin aumentar N.
- **El notch solo no basta.** En P2 y P3 su ΔSNR fue 0 dB porque no hay un tono que anular. Se agregó el FIR pasabajos (corte 300 Hz sobre la banda útil) y ΔSNR subió a 46.2 dB en P2; su retardo de 31 muestras se compensa al evaluar.
- **Tiempo real sin mirar hacia adelante.** El método espectral de la Tarea 1 iniciaba la estimación del ruido con muestras futuras; el Nodo 3 la inicia solo con el primer bloque (la diferencia con la Tarea 1 cae de 2×10^{-1} a 5×10^{-8} en 6 s). El primer bloque del Nodo 1, cuya ventana es mitad ceros, no estima, porque daba una detección falsa.
- **Métricas en el lazo acústico.** El parlante, la sala y el micrófono agregan deriva de reloj, ganancia desigual y desfase, así que las métricas de forma de onda solo son orientativas: se usan la atenuación medida en frecuencia, la energía por banda y una ganancia del trayecto ajustada por mínimos cuadrados.
- **Recursos de la Pico.** Las latencias de 85, 103 y 257 ms superan los 31.7 ms de un bloque, así que el sistema graba hasta 2 s (límite de memoria) y luego procesa: tiempo cuasi real. Se libera memoria antes de cada FFT porque `ulab` necesita bloques contiguos de 8 KB. La salida va por USB a la PC (aprobado por el profesor) en lugar de PWM, y los tres nodos corren en una sola Pico [COMPLETAR: motivo].
- **CRC en MicroPython.** Como no tiene `binascii.crc_hqx`, el firmware usa una tabla CRC-16 CCITT propia, idéntica a la de la PC; en hardware no hubo errores de CRC ni tramas perdidas.

Correcciones a propuestas de la IA. (1) La primera simulación ubicaba las interferencias justo sobre un bin, lo que daba error de estimación 0 y no probaba la interpolación; se cambió a muestrear a la f_s real medida. (2) Las pruebas de casos borde revelaron la detección falsa del primer bloque descrita arriba. (3) Los mensajes con "ñ" fallaban en la consola de Windows (codificación cp932) y la variable `PYTHONUTF8` propuesta no funcionó en PowerShell; se dejaron mensajes en ASCII.

Declaración del uso de inteligencia artificial

Se usó el asistente Claude (Anthropic) por chat para: (i) explicar conceptos (*overlap-add*, interpolación parabólica, frecuencia aparente); (ii) guia en la propuesta de la simulación de los tres nodos y el script de métricas de la Tarea 1.